Finding Signatures to Detect ICS Honeypots in the Wild

Group 8

Arul Agrawal
Delft University of Technology

Andrea Malnati
Delft University of Technology

Mitchell van Pelt
Delft University of Technology

Mrityunjaya Palanimurugan Vasanthakumari
Delft University of Technology

Erin Vergeer
Delft University of Technology

ABSTRACT

Industrial Control System (ICS) honeypots mock ICS protocols to collect intelligence and distract attackers from targeting real exposed ICS devices. The level of security provided by these honeypots depends on their indistinguishability from real devices. While most ICS honeypots are high-interaction and thus hard to distinguish, previous works have shown that they can automatically be identified trough detecting a signature in the interaction fingerprint. In this paper, we dive deeper into this technique and introduce our method for systematically identifying signatures in honeypot open source code. We present fourteen new signatures for five ICS honeypots and introduce a system for categorising them. Our evaluation shows that most of the signatures are effective in identifying honeypots out of random hosts on the internet. We conclude that the signature technique is a powerful tool for uncovering high-interaction honeypots that can be applied to any honeypot with available source code.

1 INTRODUCTION

Industrial Control System (ICS) devices are digital compute modules used for regulating industrial machinery. These include programmable logic controllers (PLCs), Supervisory Control and Data Acquisition (SCADA) Systems and Distributed Control Systems (DCS). ICS devices can be used to monitor and control a wide variety of industrial apparatuses, ranging from individual machines in factories to whole networks of equipment, such as electrical grids.

Many ICS devices can be remotely controlled by interaction through ICS protocols, which are build upon the TCP/IP (or UDP) stack. Virtually all of these protocols are designed solely from a perspective of functionality, with little to no thought going into security. This has resulted in the protocols being plaintext and with no authentication, which means that anyone who can connect to a device can control it. Thus, the only way to protect ICS devices from unauthorised access is to set them up in a secured network, unreachable from the public internet. Despite this being an obvious best-practise, there are still ICS devices that are publicly accessible and therefore vulnerable to manipulation by malicious parties. Considering that these devices are also used to control societally critical infrastructure, such as power grids and flood defences, the potential impact of a large scale cyberattack involving ICS devices cannot be underestimated. This is especially relevant in the current geopolitical climate, where hybrid warfare involving cyberspace and the internet of things is becoming increasingly more common.

Cybersecurity defences can mitigate the risks associated with vulnerable ICS devices. The most straightforward solution is to remove ICS devices from the public internet. However, this can only be done by the operators of the devices. A risk mitigation strategy that is being implemented by the wider cybersecurity community is the deployment of ICS honeypots. These are hosts pretending to be ICS devices exposed on the internet. They distract attackers from interacting with real devices and are a valuable source of threat intelligence. Honeypots can be low- or high-interaction. Low interaction honeypots only have open ports without real services or only establish a basic connection. High-interaction honeypots go a step further and will attempt to mock the real protocol. High-interaction honeypots are thought to be effective in fooling attackers, because automated scanning cannot reveal them as honeypots.

There exist many high-interaction ICS honeypots for a wide spectrum of protocols. They are thought to be effective in fooling attackers and therefore widely deployed. However, recent work by Mladenov et al. has shown that many ICS honeypots might not be as stealthy as they are believed to be [7]. Their research into ICS honeypots uncovers that different indicators can be used to construct an accurate picture on whether an ICS device on the internet is a honeypot. These include the number of open ports and the organisation hosting the device. However, the even stronger honeypot indicator presented by Mladenov et al. is the fingerprint signature. They found that popular honeypots for the S7 and ATG ICS protocols have a distinct fingerprint, called a signature. By simply sending a request to a host that returns this fingerprint, one can easily confirm the host is a honeypot if the signature is present. This opens up the possibility to trivially label many ICS protocol hosts as honeypots using automated internet scanning.

The advantage of automatic honeypot identification via signature extraction is that it makes it more feasible for security researchers to construct a list of potentially real exposed ICS devices. Finding real devices and informing their operators is an essential step in reducing the number of vulnerable ICS systems on the internet.

While there are many ICS protocols and many distinct honeypots for them, Mladenov et al. only identify signatures for two protocols and three honeypots. In this work, we answer the question: **What fingerprint signatures can be identified in the protocols mocked by popular ICS honeypots?**

We present our method for systematically extracting signatures from open source honeypot source code and share fourteen new signatures for identifying five popular ICS honeypots. This method is not only relevant to ICS and can be applied to any type of honeypot for which source code is available. We contribute to the knowledge of honeypot signatures, mainly by introducing a system for dividing them into three categories: configuration signatures, partial implementation signatures, and implementation error signatures. By trying to find our signatures in fingerprints from 52,446 random

ICS protocol hosts on the internet, we evaluate their effectiveness in identifying honeypots. This evaluation shows that almost all of our signatures appear in the wild, with some being found on hundreds of devices.

In the second chapter of this paper, we first discuss relevant background on ICS devices and honeypots, as well as related a deeper dive into related work. Then, in chapter 4, we explain our method and the found signatures. In chapter 5 we present our evaluation of the signatures. Finally, in chapter 7 we discuss our findings and we sum up our conclusions in chapter 7.

We have made our code used for the evaluation publicly available so our results can be reproduced. The code forms a specialised and user-friendly Python tool that can be used to test for the signatures on one host or a batch of IPs, and is easily extendible with new signatures. It can be found on https://github.com/Mitchell-24/Honeypot-Detector.

2 BACKGROUND

2.1 Industrial Control Systems

Industrial Control Systems (ICS) are specialized networks of hardware and software designed to monitor and automate critical physical processes such as regulating fuel levels at service stations, maintaining voltage on power grids, or controlling assembly lines in manufacturing plants. Unlike general IT systems, ICS devices (PLCs, RTUs, DCS controllers) interface directly with sensors and actuators in the physical world, making high availability and deterministic real-time performance paramount.

Many widely deployed ICS protocols were developed decades ago with reliability and real-time constraints in mind, rather than security. Well known examples include S7, Modbus, DNP3, IEC 60870-5-104 and ATG.

None of these protocols include native authentication or encryption, so a device speaking Modbus/TCP or IEC 104 can be queried or manipulated by any network-connected actor.

Traditionally, ICS networks were isolated from external access to ensure safety and reliability. However, the growing demand for remote monitoring, cloud integration, and cost-effective maintenance has led to a sharp increase in Internet-visible ICS endpoints. In April 2015, approximately 40,000 exposed ICS devices were reported by Shodan [6]. By January 2025, this number had grown to over 145,000 exposed ICS endpoints, according to Censys [4].

This more than six-fold increase in just ten years highlights the growing value of ICS systems as targets for attackers. A successful disruption can result in significant economic losses, safety hazards, or even large-scale blackouts. Therefore, distinguishing between genuine public-facing ICS controllers and decoy systems such as honeypots (e.g., *Conpot*, *GasPot*) has become critically important for both defenders and adversaries.

2.2 ICS Honeypots

A *honeypot* is a security resource whose value lies in being probed, attacked, or compromised. *ICS honeypots* are specialized decoy systems that emulate ICS devices such as PLCs or Automatic Tank Gauging (ATG) units to lure attackers and observe their methods. ICS honeypots can be broadly categorized into two types: low-interaction and high-interaction honeypots.

Low-interaction honeypots simulate only basic network behaviour, such as open Modbus/TCP or IEC 104 ports, and return canned or static responses without implementing the full protocol state machine. For example, a Modbus honeypot listening on port 502 may always return the same response regardless of the query. These honeypots are relatively easy to fingerprint due to missing protocol fields, identical banners, or unrealistic timing patterns.

High-interaction honeypots, on the other hand, run complete protocol stacks or actual firmware in a sandboxed environment to mimic real device communications. A good example is *GasPot*, which fully implements the ATG protocol logic used in fuel station tank gauges [11]. These honeypots are much harder to distinguish from genuine ICS devices because they exhibit realistic and stateful interactions.

The primary objectives of ICS honeypots include *traffic diversion*, *intelligence gathering*, and *security improvement*. First, they serve to divert malicious scans and exploits away from production ICS assets, effectively acting as decoys. Second, they are used to gather intelligence by recording attacker commands, payloads, and techniques for later analysis. Finally, the collected data can be used to refine intrusion detection systems, develop mitigation strategies, and harden real ICS deployments against threats.

2.3 General honeypot detection techniques

Distinguishing honeypots from genuine ICS devices is crucial for both adversaries and defenders. For attackers, the ability to detect honeypots ensures they do not waste time interacting with decoys, allowing them to focus their efforts on real, exploitable targets. For researchers and security teams, detection serves several important purposes. It enables them to assess the stealth and fidelity of deployed honeypots, to correctly interpret scanning and attack data for accurate threat modelling, and to improve future honeypot design and deployment strategies.

Several complementary techniques are commonly employed to distinguish genuine ICS devices from decoy honeypots. These methods provide different indicators of a honeypot's presence and can be combined for greater confidence.

Port Behaviour Analysis [7]: Real ICS controllers typically expose only the specific ports required by their protocol implementations (e.g., a PLC might listen exclusively on TCP/102 for S7 traffic). In contrast, honeypots often open many ICS-related ports simultaneously to emulate multiple device types and may also open more non-ICS ports. A scan that reveals an unusually large set of open ICS ports can therefore suggest the host is a honeypot.

IP and Network Metadata [9]: Many honeypots run on cloud infrastructure or research networks rather than on industrial subnets. Enriching IP addresses with ASN, organization name, geolocation, and reverse-DNS information can reveal non-industrial hosting (for instance, AWS, GCP, or generic "hosted-by" entries) that is atypical for true ICS equipment.

3 RELATED WORK

Prior research on honeypot detection and Internet-facing ICS exposure has made significant strides in uncovering both the scale of ICS exposure and the limitations of existing honeypot implementations. Four notable works in this area are by Mladenov et al., Srinivasa et

al., Williams et al., and Sun et al., each contributing distinct insights, datasets, and detection methodologies.

Mladenov et al. [7] conducted the most comprehensive Internet-scale study of ICS exposure to date. They analysed over 150,000 ICS endpoints across 17 industrial protocols using application-layer scans from Censys. Their detection pipeline combined full protocol handshakes, IP metadata enrichment (via IPinfo), and static protocol fingerprinting. For instance, they detected honeypots like GasPot using fixed timestamp responses (e.g., "9999FF1B") and identified Conpot via its hardcoded S7 diagnostic string "Mouser Factory." Additionally, they observed that honeypots frequently expose multiple unrelated ICS services simultaneously—unusual for genuine industrial deployments. Their classification framework included confidence scoring, and results were validated through controlled honeypot deployment and responsible disclosure. Their findings suggest that up to 25% of publicly reachable ICS hosts are honeypots.

Srinivasa et al. [9] proposed a general-purpose, multistage honeypot fingerprinting framework applicable to both IT and ICS services. Their methodology includes active probing (e.g., protocol-level handshakes, SSL certificate checks) and passive metadata inspection (e.g., ISP, ASN, geolocation). The system was evaluated across 2.9 billion IPv4 addresses, identifying over 21,000 honeypots, including SSH, HTTP, FTP, and ICS decoys like Conpot and GasPot. Detection was primarily based on static or anomalous response patterns and known vendor artifacts, without focus on ICS protocol state fidelity or runtime variability.

Williams et al. [12] introduced a novel passive detection technique based on timing consistency in ICMP responses. Their work, titled *Time-to-Lie*, identifies ICS honeypots by analysing consistent round-trip times (RTTs) and TTL values across multiple probes. They show that many honeypots exhibit static latency behaviour—resulting from scripting or virtualized environments—unlike real ICS devices, which display more natural timing variability. This work provides a lightweight, non-intrusive way to detect honeypots without needing to engage in full protocol communication.

Sun et al. [10] proposed a complementary active probing approach through lightweight fuzzy testing. They constructed malformed or edge-case protocol messages to elicit inconsistent or abnormal responses from ICS honeypots. Their evaluation demonstrated that many honeypots lack robust error handling and produce fixed or empty replies when presented with unexpected input—contrasting with real devices that exhibit protocol-specific error codes and state transitions. Their approach is protocol-agnostic and efficient, suitable for Internet-scale scanning environments.

Although these studies collectively offer strong foundations for identifying honeypots, several key challenges remain. Mladenov et al. and Srinivasa et al. rely heavily on static response signatures or IP metadata, which can be evaded by more sophisticated honeypots. Williams et al.'s approach is passive but limited in protocol depth, and Sun et al.'s fuzzing strategy is effective but may not differentiate high-interaction honeypots with complete state machines.

To address these limitations, our research augments existing techniques by combining static signature elicitation with state-aware protocol interaction and dynamic response timing analysis. Our methodology enables robust detection of both low- and high-interaction ICS honeypots—even when static fingerprints are missing, randomized, or intentionally obfuscated—thus improving resilience against evasive deployments and supporting more accurate classification in realistic scanning conditions.

4 METHODOLOGY

As already mentioned in Chapter 1, our goal is to automate the detection of ICS honeypots. Following the signature-based approach of Mladenov et al. [7], we achieve this by extracting protocol-level fingerprints that can be queried remotely and are not expected to be returned by a real ICS device. First, Section 4.1 formalises the signature taxonomy, Section 4.2 then details the workflow we use to uncover new signatures, and Section 4.3 catalogues the results and explains how each finding can be reproduced and why it belongs to its assigned category.

4.1 Signature Taxonomy

To keep the discussion clear, we sort every signature into one of three categories: configuration, partial implementation, and implementation error. We then formalise this taxonomy, highlighting how the three classes differ in the information they expose and in the weight of the evidence they provide.

Configuration signatures. Honeypots often come with hard-coded configuration values (e.g., serial number, device name, firmware version) that are parsed at startup and used at runtime to emulate a real device. These parameters may sit in a template file that is easy to change, or embedded in the source code, harder to modify. When the device and its protocol port are reachable from the Internet, the entire protocol is exposed, allowing remote clients to pull this metadata with a simple diagnostic request. In principle, such values should be customized before deployment; however, in many cases they are left in their original state. This makes it possible to identify the host as a honeypot, since the likelihood that a real industrial device would expose those exact same static values is extremely low. Configuration signatures therefore offer the easiest and safest method to detect a honeypot in practice.

Partial implementation signatures. Many honeypots implement only a subset of the target protocol, leaving some valid commands unhandled. When the detector sends such a request, the honeypot typically returns a generic not-supported error, closes the connection, or does not reply at all, whereas a real ICS device would answer with a proper response. This mismatch reveals the host as a honeypot and therefore constitutes a partial implementation signature.

Implementation error signatures. In this case the honeypot does implement the requested command, yet its reply deviates from the protocol specification. Typical anomalies include malformed data fields, unexpected response codes, or closing the connection after a syntactically invalid request instead of returning the proper exception and keeping the session open. A real ICS device would send a fully compliant response, or a standards-conformant error, while maintaining the connection. Detecting such protocol violations therefore clearly distinguishes implementation error signatures from partial-implementation signatures, where the command is simply unimplemented.

4.2 Approach for Finding Signatures

To discover new signatures, we propose a generalized process that can be applied to any honeypot. When source code is available, we begin with a *static analysis*: template files and constant variables are examined for hardcoded default values that, if left unchanged, would surface as configuration signatures. The request handling logic is then inspected to determine which commands are fully supported, which are missing, and how replies are constructed. These observations provide clues to partial implementations or implementation errors. Whenever static analysis uncovers candidate signatures, we launch the honeypot locally and issue diagnostic or identification commands to retrieve those fields, thereby confirming the configuration signature. Likewise, if a function appears missing in the code, we send the corresponding request and compare the response with the protocol specification to validate a partial implementation signature.

If during inspection of code no clear indication comes up, we proceed with a *black-box interaction*. In this phase, we interact with the honeypot by sending a mix of valid and deliberately malformed requests; unhandled commands expose partial implementation signatures, while malformed or non-standard replies reveal implementation error signatures. During local tests, we imposed no restriction on the request set in order to maximise coverage. When operating in the wild, however, we limited ourselves to read-only or otherwise non-mutable requests in order to avoid altering the state of real ICS devices.

4.3 Found signatures

We provide a detailed overview of the honeypot signatures uncovered during our analysis of five different honeypots and summarized in Table 1, explaining how each one can be reproduced. This includes both newly identified signatures and previously known ones, which we have independently confirmed to still be present in recent honeypot deployments.

Conpot S7. The S7 protocol emulated by Conpot exposes both configuration and partial implementation signatures. The configuration signature had already been documented in prior studies [7], and we confirmed that it remains present in current versions of Conpot. It can be retrieved by sending a diagnostic request, after completing the standard ISO-on-TCP and COTP handshakes. If the response includes the static strings “Mouser Factory” and “88111222”, the default values embedded in the honeypot’s configuration, we detect the presence of the signature.

To reproduce the partial implementation signature, we again initiate a session and send a valid diagnostic command, which should return a legitimate response. We then issue a standard read request, which is not implemented in Conpot’s S7 stack. As a result, the honeypot closes the TCP connection immediately. To verify this, we resend the same diagnostic request as before: a real device would reply again, while Conpot remains silent due to the closed connection. This behavioural mismatch confirms a partial implementation signature. Notably, the same scenario is observed with several other valid S7 functions(e.g., write, upload).

Conpot/Gaspot ATG Conpot and GasPot share the same implementation of the ATG protocol [1], and therefore expose identical signatures. The configuration signature can be retrieved by sending the command `I20100`. The response includes static values such as the banner “\n\n\n\n”, the string “Gilbarco”, the keyword “Serial”, and a timestamp formatted as “MM/DD/YYYY HH:MM”. The banner and timestamp format were already known from prior analyses and are still present in the current implementation.

In addition, the honeypot exhibits a partial implementation signature: when sending valid commands such as "I30100", it replies with the fixed string “9999FF1B”, which represents a generic unsupported-command message. This behaviour is not compliant with the ATG protocol, as documented in prior work [8], and our testing confirms it remains unchanged.

Conpot IEC-104 The Conpot implementation of the IEC-104 protocol exposes a configuration signature in response to a general interrogation request. If the reply contains station address "7720" and exactly "61" data points, of which 59 are non-empty, the signature is considered present.

Conpot BACnet The BACnet protocol implementation in the official version of Conpot is currently not functional [2]. It responds to any incoming request with the message “Decoding Error – PDU: \<PDU None → None : …”, regardless of the query type, revealing an implementation error signature.

However, a community-maintained fork of Conpot [3] provides a working BACnet version, which makes it possible to extract a configuration signature through a series of "readProperty" queries. Specifically, by requesting standard BACnet fields such as objectIdentifier, vendorId, vendorName, modelName, and objectName, the device returns static values like Vendor “Cornell University (15)”, Vendor Name “Alerton Technologies, Inc.”, Object Identifier 36113, Object Name “SystemName”, and Model “VAV-DD Controller”.

Conpot IPMI The Conpot implementation of the IPMI protocol exposes a configuration signature that can be retrieved by issuing an “mc info” request. The response consistently contains the same hardcoded values: Device ID "37", Device Revision "0x13" or "19", IPMI Version "2", and Manufacturer ID "15".

In addition, an implementation error signature can be observed when attempting to issue a warm reset using the command “mc reset” warm. While the cold reset variant is supported, the warm version fails with the message "MC reset command failed: Invalid command".

Conpot Modbus The Modbus implementation in Conpot exposes a configuration signature that can be retrieved by issuing a Device Identification request. The response includes static strings such as "Siemens", "SIMATIC", and "S7-200", along with fixed protocol fields like MEI type 14 and conformity level 1.

In addition, previous studies [8] have identified an implementation error signature triggered by sending the command `00 00 00 00 00 05 00 2B 0E 02 00`, which corresponds to a broadcast Device Identification request. Although such broadcasts are not allowed by the Modbus specification, a real device would silently discard the message while keeping the connection open. Conpot, instead, closes the TCP connection. This behaviour was verified locally but was not included in our experiments due to its unreliability and high false positive rate.

Conpot EtherNet/IP The Conpot implementation of the EtherNet/IP protocol exposes a configuration signature in response to a List Identity request. The device returns static metadata such

as product name "1756-L61/B LOGIX5561" and the serial number "7079450", which match the default configuration of the honeypot.

Conpot SNMP The SNMP protocol in Conpot exposes a configuration signature retrievable by issuing a standard SNMP walk under the system subtree (OID 1.3.6.1.2.1.1). The response contains static values such as "Siemens, SIMATIC, S7-200", "Siemens AG", "CP 443-1 EX40", and "Venus", along with constant numeric fields like "72", "0", and sysObjectID "1.3.6.1.4.1.20408".

DNP3pot DNP3 The DNP3pot honeypot does not implement any actual DNP3 commands. Instead, it replies to every request with the same generic static response frame: `05 64 0A 00 01 00 05 00 39 71 C0 F4 82 00 00 6F BA`. This behaviour reveals a partial implementation signature, as all requests trigger the same placeholder reply.

MedPot HL7 While not strictly ICS, we also investigate HL7 since it is also used in societal critical settings. The MedPot's HL7 protocol exposes one signature that sits between the configuration and implementation error classes. The static ACK banner shown below comes from its default configuration (template file), yet it is returned because an implementation flaw accepts any frame that begins with `MSH|` as valid. Removing the signature entirely would therefore require fixing the implementation and not just editing the template before deploying the honeypot.

```
MSH|^~&|SENDING_APPLICATION|SENDING_FACILITY|
RECEIVING_APPLICATION|RECEIVING_FACILITY|20110614075841||
ACK|1407511|P|2.3||||MSA|AA|1407511|Success||
```

Dicompot DICOM The Dicompot honeypot exposes a configuration signature in response to a standard C-ECHO request. During association, it reveals static implementation metadata including the Implementation Version Name "dicompot", the SCP Title "storescp", and support for the Verification SOP Class (UID 1.2.840.10008.1.1). While also not strictly ICS, we include this protocol for the same reason as HL7.

5 DETECTING ICS HONEYPOTS IN THE WILD

The signatures that we identified are all confirmed to work for local sandboxed implementations of the honeypots. To find out how effective the signatures are in a real-world scenario, we evaluate their effectiveness in identifying honeypots on real hosts on the internet.

5.1 Implementation and Experimental Setup

5.1.1 ***Censys Data Acquisition.*** To evaluate the effectiveness of our honeypot detection signatures in real-world environments, we conducted a comprehensive analysis using data collected from Censys, a prominent internet-wide scanning platform [5]. We utilised the Censys Command Line Interface (CLI) to collect data on hosts running ICS protocols on the internet. Our data collection aimed to determine the canonical port number(s) for each of the ICS protocols considered.

We limited our collection to 75 pages per protocol to ensure a representative sample whilst maintaining manageable dataset sizes. This approach provided comprehensive metadata for each host, enabling both our signature-based detection and comparative analysis with Censys' own honeypot classifications.

5.1.2 ***Scanning Procedure.*** We developed a Python-based detection tool that systematically tries to match the signatures identified in our methodology to determine whether a given host is running a honeypot implementation. The tool operates by connecting to target hosts on their open ports, attempting to elicit the specific response patterns (fingerprints), match these with known signatures, and classify hosts based on the signatures detected.

When multiple signatures are identified on a single host, the tool determines which specific honeypot software is likely being deployed. This automated approach enables large-scale analysis of the hosts collected from Censys data.

In order to prevent accidental damage to real systems, our tool does not test any signatures that involve executing any operations which could alter the state of a real system.

5.1.3 ***Data Processing and Analysis.*** Censys provides its own honeypot classification tags for scanned hosts. We leverage this metadata to compare our approach against Censys's methodology. Our analysis focuses on three critical scenarios:

(1) Hosts identified as honeypots by both our tool and Censys
(2) Hosts identified as honeypots by our tool but not by Censys
(3) Hosts identified as honeypots by Censys but not by our tool

Through analysis of these three cases, we aim to identify potential weaknesses in both our approach and Censys's methodology. Cases where only one system identifies a honeypot are of particular interest, as they indicate scenarios where either tool improvements are possible or false positives have occurred. In both instances, such analysis enables refinement of the respective detection systems.

5.2 Results

Among the scanned 52,446 hosts, a total of 800 signatures (from DICOM, ATG, IEC104, DNP3, HL7, S7, ENIP and IPMI) were found in 508 distinct hosts. No signatures were detected for other protocols we considered (MODBUS, BACNET and SNMP) in our search. For convenience, we refer to detected signatures by their given protocol name in the remainder of the section. For S7, the two distinct signatures are represented as S7-1 and S7-2 for the configuration type signature (CFG) and partial implementation (PI) type signature, respectively.

Please see table 2 for a summary of all detected signatures from protocols that were considered in the experiment. The top 2 signatures (DICOM & ATG) account for more than half of all signatures, which is a sign they are popular target protocols for honeypots. Surprisingly, we can observe a difference in the number of signatures found for S7-1 and S7-2. More signatures are detected for S7-2, which may indicate the presence of honeypots that have their default configuration adjusted for stealth.

5.2.1 ***Honeypot labels.*** We cross-reference the findings with the Censys data, which has the number of open ports and its own label for if a host is thought to be a honeypot. Preliminary analysis reveals that 75 out of the 508 hosts on which one or more of our signatures were found have the honeypot label present (see figure 1). We briefly provide two perspectives on the observed lack of honeypot labels.

First, we hypothesise that in many cases Censys does not provide honeypot labels where they are expected. Although Censys uses a

Table 1: Honeypot signatures considered in this study. CFG = configuration, PI = partial implementation, ERR = implementation error.

Honeypot	Protocol	Class	Signature
Conpot	S7	CFG	Plant ID “Mouser Factory”
Conpot	S7	CFG	Serial # “88111222”
Conpot	S7	PI	`Read/Write` and other functions not implemented
Conpot/Gaspot	ATG	CFG	Banner contains “\n\n\n\n”
Conpot/Gaspot	ATG	CFG	Timestamp format “MM/DD/YYYY HH:MM”
Conpot/GasPot	ATG	CFG	Banner contains “Gilbarco”
Conpot/GasPot	ATG	CFG	Banner contains “Serial”
Conpot/GasPot	ATG	PI	Replies with “9999FF1B” to known commands
Conpot	IEC-104	CFG	Station addr 7720; 61 endpoints (59 non-empty)
Conpot	BACnet	ERR	“Decoding Error – PDU: \<PDU None → None : . . . ”
Conpot (fork)	BACnet	CFG	Vendor “Cornell University (15)”, Vendor Name “Alerton Technologies, Inc.”, Obj-ID 36113, Object Name “SystemName”, Model “VAV-DD Controller”
Conpot	IPMI	CFG	Device ID “37”; Device Revision “0x13” or “19”; IPMI Version “2”; Manufacturer ID “15”
Conpot	IPMI	ERR	Returns “Invalid command” to commands: “mc reset warm”
Conpot	Modbus	CFG	Device identification contains “Siemens”, “SIMATIC”, “S7-200”; MEI type = 14; Conformity level = 1; Read Coils returns constant value
Conpot	Modbus	ERR	TCP connection closed after command “000000000005002B0E0200”
Conpot	EtherNet/IP	CFG	Product Name “1756-L61/B LOGIX5561”, Serial # "7079450"
Conpot	SNMP	CFG	sysDescr contains “Siemens, SIMATIC, S7-200” and “CP 443-1 EX40”; vendor “Siemens AG”; host name “Venus”; constants “72” / “0”; sysObjectID 1.3.6.1.4.1.20408
DNP3pot	DNP3	PI	Replies with “05640A00010005003971C0F48200006FBA” to known commands
Medpot	HL7	CFG	Returns fixed ACK: “MSH|^~&|SENDING_APPLICATION|... |MSA|AA|1407511|Success||”
Dicompot	DICOM	CFG/ERR	Implementation Version “dicompot”, SCP Title “storescp”, Verification SOP Class 1.2.840.10008.1.1

Table 2: Signatures detected in 508 hosts (among 52,446 hosts)

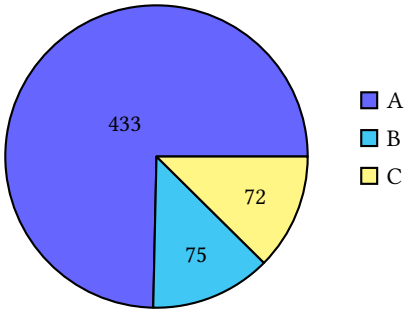
Signature	Hosts with signature	Share of total signatures	Share of signature hosts	Share of total hosts
DICOM	291	36.4%	57.3%	0.55%
ATG	249	31.1%	49.0%	0.47%
IEC104 (Conpot)	111	13.9%	21.9%	0.21%
DNP3	89	11.1%	17.5%	0.17%
HL7	28	3.5%	5.5%	0.05%
S7-2 (Conpot)	14	1.75%	2.8%	0.03%
ENIP (Conpot)	14	1.75%	2.8%	0.03%
S7-1 (Conpot)	3	0.38%	0.59%	<0.01%
IPMI (Conpot)	1	0.13%	0.20%	<0.01%
Total	**800**	**100% (800)**	**100% (508)**	**0.97% (508)**

wide range of methods to determine whether a honeypot is likely, Censys may have been unable to conclude a host is a honeypot based on these methods alone. Another observation is that the labels do not seem to be consistent across the different data files. We discover that in the Censys data, hosts that match the ATG service, not one host is labelled as a honeypot. While some of these hosts are actually present in other capture files for other services, where they *do* have the honeypot label. This leads us to believe that many of the ATG hosts in the concerning Censys data wrongfully do not have the honeypot label.

Another reason for the seemingly missing labels is that for part of the hosts we may have wrongfully concluded the presence of a signature (false positive). Concretely, some signatures are based on whether a host responds or not. It is possible that such a signature is mistakenly and accidentally triggered by other protocol servers for which the signature is not intended.

5.2.2 ***Concurrent honeypots and open ports.*** More often than not, servers running honeypots also run a lot of other services, potentially even other honeypots. This is seen in [7]. We can use this knowledge to further investigate hosts with our signatures

Indeed A + B = 508 total hosts have one or more of our signatures. C represents honeypots we could not identify.



- **A** = Hosts with at least one signature but without Censys honeypot label
- **B** = Hosts with at least one signature *and* Censys honeypot label
- **C** = Hosts without signatures but with Censys honeypot label

Figure 1: Categorizing honeypot hosts across all protocols.

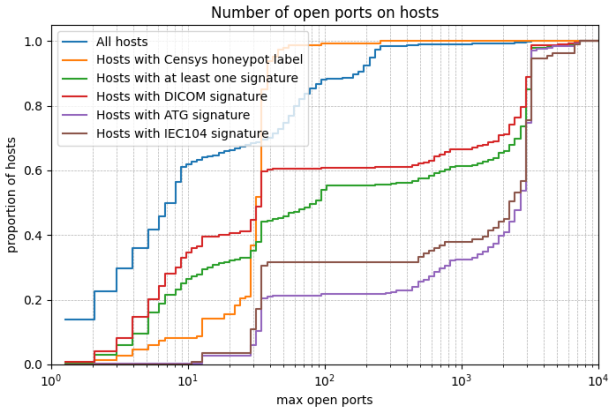


Figure 2: Visualization of open port count according to the provided Censys data. Each graph is plotted with 100 data points spaced evenly on the logscale.

but without Censys honeypot labels, of which there are many (see category A in figure 1). This way, we attempt to confirm with a higher level of certainty whether the findings are valid. More precisely, we attempt to confirm whether the found signatures rightfully matched on their associated hosts.

A plot is attached (see figure 2) to show the proportion of hosts with the number of open ports below a corresponding max value. For reference, we have a graph to show the number of open ports among all hosts, as well as a graph to show the number of open ports of hosts with Censys labels. Additionally, we have plotted a graph for the hosts with at least one of our signatures and graphs for hosts on which the top 3 signatures (DICOM, ATG, IEC104) are detected. From the figure we can conclude that a majority of likely honeypots have ten or more open ports, while this is not the case for all hosts. Moreover, the graphs show that the number of open ports for hosts with our signature is high. The results for ATG and IEC104 signature hosts show that the number of open ports is especially high, and this is also found in [7].

We now present the exact number of hosts running two or more honeypot instances concurrently, for those hosts with at least one of our signatures (category A and B). Let us look at category A and B separately. Among the 433 hosts in category A, which are the hosts with at least one signature but without labels from Censys, we have found 130 hosts with two or more signatures. These 130 hosts all involve a pair or triple of DICOM, ATG and IEC104 honeypots. Based on these results, we classify these 130 hosts as honeypot hosts with a high level of certainty, even though they have not been labelled as such by Censys. It also supports the statement that the DICOM, ATG and IEC104 signatures are in fact valid and not false positives. Lastly, we turn to category B, which are the hosts on which one of our signatures was found and in addition have the Censys label. 65 out of 75 hosts in this category run two or more honeypot instances concurrently. The hosts seem to involve HL7, ENIP and S7, in addition to DICOM, ATG and IEC104, sometimes running all six instances concurrently.

5.2.3 ***(Potentially) non-identified honeypots.*** Among hosts with Censys labels, on 72 of them we did not find any of the considered signatures. These hosts were found to run services on ports associated with DICOM (11112/tcp), ENIP (44818/tcp), HL7 (2575/tcp), S7 (102/tcp), IEC104 (2404/tcp) and SNMP (161/udp). Most of them in fact run/emulate DICOM or IEC104, according to Censys. Why then, did these hosts not match any of the signatures? The most plausible explanation is that both signatures, as is the case for most of our signatures, are configuration based. It is possible to remove these signatures from the fingerprint by changing the configuration, and we assume that experienced network engineers make sure to customise the configuration before deployment. This possibility is supported by the fact that we did not miss any ATG signatures among the honeypots recognized by Censys. After all, the partial implementation on which the ATG signature is based cannot be readily fixed and is thus more likely to persist in the wild, increasing the reliability of the signature. The same is also true for DNP3, which is based on partial implementation. All Censys-labelled DNP3 honeypots were found by us and none were missed.

The remaining anomaly is the hosts in category C which run/emulate S7, according to Censys, of which we found three. If they really are honeypots, we should have found our partial implementation signature on these hosts with high probability. However, we did not find any signatures. Possibly, these hosts ran honeypots at the time of capture, but no longer at the time of the scan attempt. We find this to be an unlikely and unsatisfying explanation, but it has to be considered. Second, we could be dealing with a more advanced and custom S7 honeypot not based on Conpot, one that has the partial implementation fixed. Third, we propose that Censys mistakenly labelled the hosts as honeypots. The hosts may have exhibited behaviour deemed suspicious by Censys, but which happens to be nominal in this particular case.

6 DISCUSSION

Limitations. The main limitation of our research is that we do not have any real ICS devices available to us for testing. This means that we are not able to verify that the signatures we uncovered in the honeypots are completely unique to the honeypot and can never be found on a real host running the protocol. However, for constructing the signatures, we consulted the specifications of the protocols and mostly focus on configuration signatures, which are unique to the honeypots. Therefore, we have good confidence that the signatures can only be found in honeypot fingerprints, and for most of the investigated signatures, the presence of signatures provide high certainty that the concerning service is running a honeypot. For a small subset however, especially signatures relating to timing-based recognition techniques or non-response type reply signatures, there is a possibility for false positives.

Moreover, another limitation is that the found signatures that rely on a function that is mutating, e.g. the IPMI implementation error signature which attempts to reset the host, cannot be evaluated on random hosts, because it would affect the operation of real devices.

Application outside ICS. In our research, we focus on ICS protocols because real devices exposing these pose a great treat to society and thus high interaction honeypots mocking them are an important defence. Our approach, however, does not rely on any specifics related to ICS. The method involves analysing source code for useful signature patterns, which relate to a broad spectrum of interaction fingerprints ranging, from hardcoded values to implementation mistakes. We believe these can be present in any type of high-interaction honeypot, and therefore our method can be generalised and applied to many other honeypots. We think that finding signatures in any type of honeypot, ranging from amateur at home honeypots to professional ones in critical infrastructure, can benefit security consciousness and serve as input for improving undetectability.

Recommendations. A core assumption about honeypots is that attackers can not easily distinguish them from real devices. Our research once again shows that for many popular ICS honeypots this is not necessarily true and that the effectiveness of the investigated high-interaction honeypots is limited. This knowledge is likely not unknown to attackers and therefore naivety about the true stealthiness of honeypots poses a risk to security.

We therefore urge anyone deploying honeypots for network security purposes to be aware of the stealth limitations and take action to improve undetectability. As we have shown, the easiest way to uncover a honeypot via a signature is by detecting some default configuration. We therefore underline the importance of customising all relevant configuration when deploying a honeypot. Furthermore, we call upon honeypot developers to do more to improve stealthiness. A simple yet effective upgrade is to create a mechanism that randomises configuration dynamically. Also, we highlight the importance of mocking a protocol completely and correctly to avoid easily detectable partial implementation or implementation error signatures.

7 CONCLUSION

Honeypot signatures are patterns or static values present in interaction fingerprints that reveal a device is a honeypot if found. The presence of signatures affects the effectiveness of honeypots and thus poses a security risk, especially for industrial control system (ICS) honeypots, whose presence on the internet functions as an important defence in protecting real exposed ICS devices.

In this work, we analyse five ICS honeypots mocking a combined eleven individual protocols. In doing so, we create a method for systematically identifying patterns in source code that can be used as honeypot signatures. We discover that the honeypots have many patterns that can be used as signatures, and that these can be grouped into three distinct categories. The first and easiest to identify type of signature is the configuration signature, which is a value or string that can be retrieved through the protocol, and is unique to the honeypot and present by default, because it has been hardcoded in a configuration file or directly into the code by the developer. Secondly, there are partial implementation signatures. These come from a honeypot not mocking the complete protocol and not responding to some types of requests that should work for a real implementation of the protocol. Lastly, there are implementation error signatures, which are mistakes in the honeypot's implementation of the protocol, likely not found in real devices as this would affect their operation.

In total, we present fourteen new signatures for five investigated ICS honeypots and confirm an additional six uncovered in other works. Of the fourteen, nine are configuration, two are partial implementation, and three are implementation error signatures.

To evaluate the effectiveness signatures, we test the fingerprints of 52,446 hosts on the internet with at least one of the eleven protocol ports open. Of the twelve signatures we test, nine are found in the wild, with some being encountered hundreds of times. Signatures based on configuration are the most common, and those based on implementation less so. When comparing the hosts with at least one signature to data from Censys, we find that many of these are not labelled by Censys as honeypots. While a small part of these might be false positives, the ascertainment of two or more different signatures on many of these hosts provide confidence that they are indeed honeypots. We therefore conclude that the signature method is more effective in identifying honeypots than the traditional methods of counting open ports and analysing host metadata. We also conclude that signatures based on implementation are a more reliable indicator than those based on configuration

While there is high confidence that the presented signatures are unique to honeypots, there is still a minor possibility of finding false positives. Therefore, future work should explicitly test the signatures on real ICS devices to confirm their effectiveness in distinguishing between real and honeypot. On the other hand, signatures for which few or none honeypots are found in the wild should be re-evaluated with a larger sample of hosts and possible upgraded to make them more effective.

Furthermore, we recommend that more research takes place into the weaknesses of high-interaction honeypots in general. Our method is not specific to ICS protocols and can be applied to any kind of honeypot. Future works could apply our signature finding method to discover signatures for other protocols and honeypots.

REFERENCES

[1] 2015. *Gas Tank Monitoring System Honeypot.* https://www.honeynet.org/2015/09/09/gas-tank-monitoring-system-honeypot/ Accessed on 2025-06-23.

[2] 2022. BACnet not working properly. https://github.com/mushorg/conpot/issues/572 Accessed on 2025-06-23.

[3] 2022. Conpot BACnet Fix Branch. https://github.com/mintos5/conpot/tree/bacnet_fix Accessed on 2025-06-23.

[4] Censys. 2024. Censys Data Reports Over 145,000 Exposed ICS Services Worldwide. https://industrialcyber.co/industrial-cyber-attacks/censys-data-reports-over-145000-exposed-ics-services-worldwide-highlights-us-vulnerabilities/.

[5] Zakir Durumeric, David Adrian, Ariana Mirian, Michael Bailey, and J. Alex Halderman. 2015. A Search Engine Backed by Internet-Wide Scanning. In *22nd ACM Conference on Computer and Communications Security.*

[6] John Matherly. 2015. State of Control Systems in the USA. https://blog.shodan.io/state-of-control-systems-in-the-usa-2015-05/.

[7] Martin Mladenov, Laszlo Erdodi, and Georgios Smaragdakis. 2025. All that Glitters is not Gold: Uncovering Exposed Industrial Control Systems and Honeypots in the Wild. In *IEEE European Symposium on Security and Privacy (EuroS&P) 2025.* Venice, Italy.

[8] Shreyas Srinivasa, Jens Pedersen, and Emmanouil Vasilomanolakis. 2021. Gotta catch 'em all: a Multistage Framework for honeypot fingerprinting. (09 2021). https://doi.org/10.48550/arXiv.2109.10652

[9] Shreyas Srinivasa, Jens Myrup Pedersen, and Emmanouil Vasilomanolakis. 2021. Gotta catch 'em all: a Multistage Framework for honeypot fingerprinting. arXiv:2109.10652 [cs.CR] https://arxiv.org/abs/2109.10652

[10] Yanbin Sun, Xiaojun Pan, Chao Xu, Penggang Sun, Quanlong Guan, Mohan Li, and Men Han. 2020. Identifying Honeypots from ICS Devices Using Lightweight Fuzzy Testing. *Computers, Materials & Continua* 65, 2 (2020).

[11] Kyle Wilhoit and Stephen Zettl. 2015. The Little Pump Gauge That Could: Attacks Against Fuel Monitoring Systems. In *Black Hat USA.* https://documents.trendmicro.com/assets/wp/wp_the_gaspot_experiment.pdf.

[12] Jacob Williams, Matthew Edwards, and Joseph Gardiner. 2024. Time-to-Lie: Identifying Industrial Control System Honeypots Using the Internet Control Message Protocol. arXiv:2410.17731 [cs.CR] https://arxiv.org/abs/2410.17731